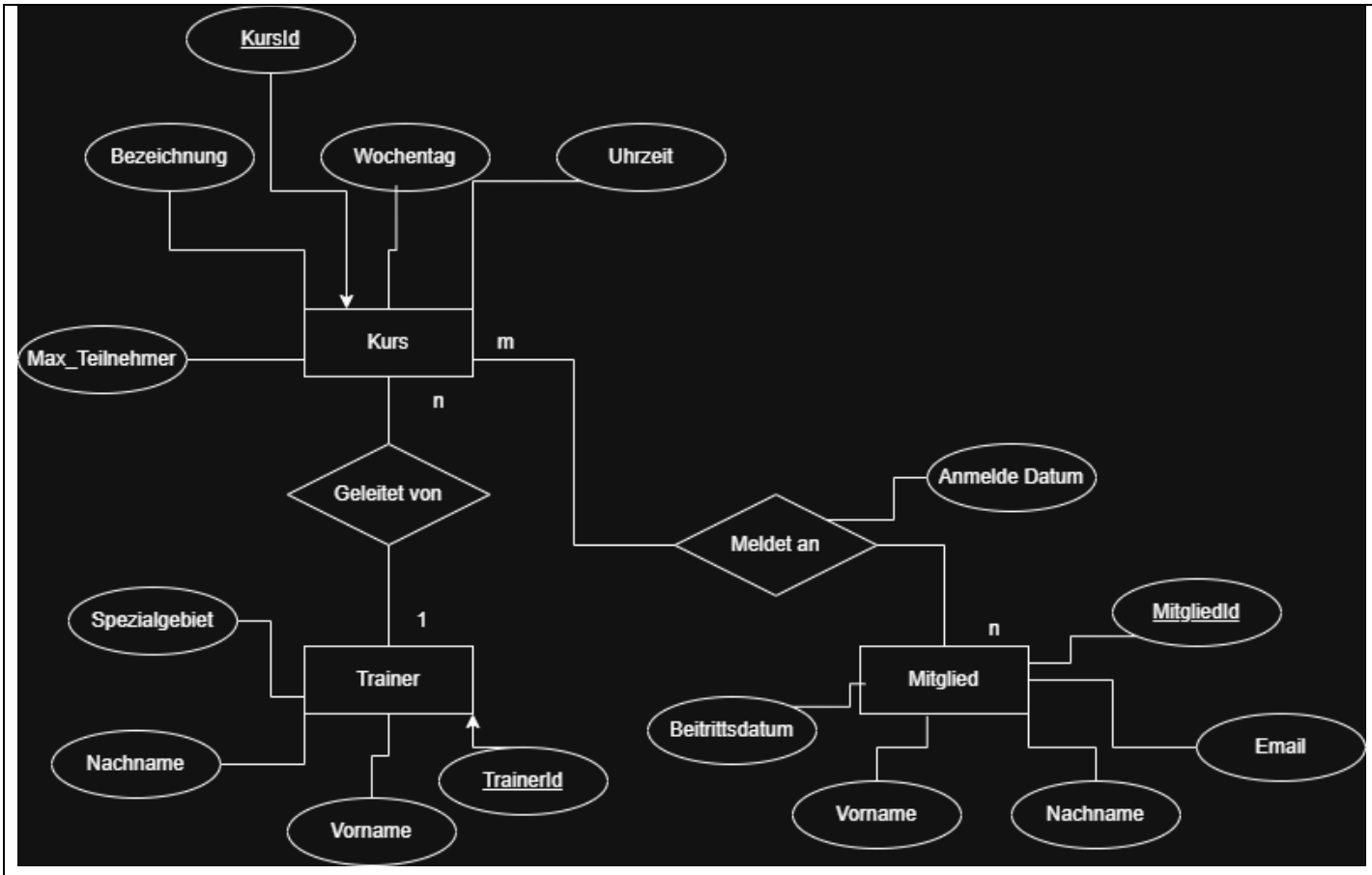
Name: Emir Alici **Datum:** 08.04.2026 **Klasse:** 3AHIF

3. Grafische Leistungsfeststellung – DBI1
Thema: Fitnessstudio-Verwaltung (ERM, RM, Python/SQLite3, Anvil)

Punkte	**/ 100**				
Note	100-87.5 □ Sehr gut	< 87.5-75 □ Gut	< 75-62.5 □ Befriedigend	< 62.5-50 □ Genügend	< 50 □ Nicht Genügend
Unterschrift					
GitHub-Link	https://github.com/ea5453/fitnessstudio-verwaltung				
Anvil Publish-Link	https://spanish-extroverted-tapaculo.anvil.app				

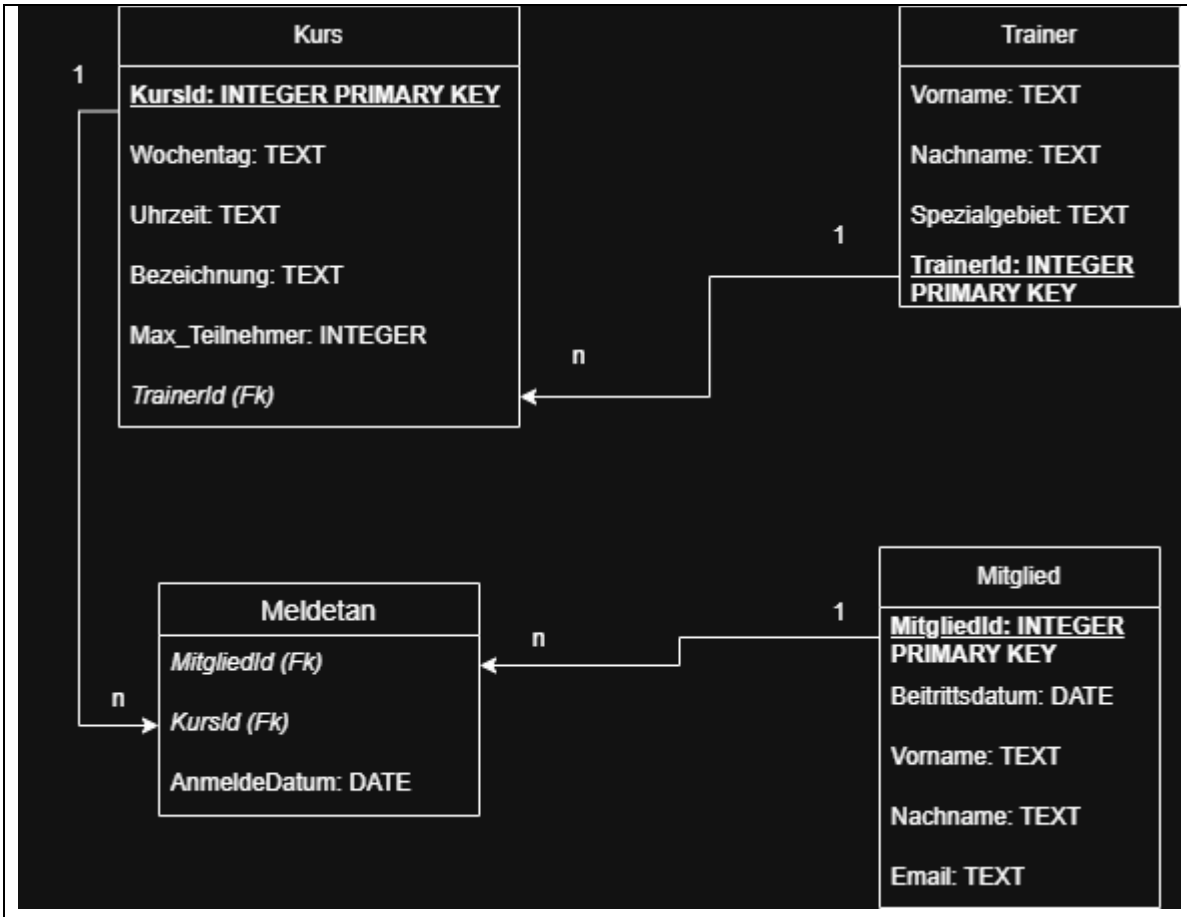
Form	**Pun**
1. Form der Arbeit (Leserlich, vollständige Abgabe, sinnvolle Dateinamen, Ausführbar, Fehlerbehandlung)	
Szenario: Fitnessstudio-Verwaltung	
Ein lokales Fitnessstudio möchte seine Mitgliederverwaltung und Kursbuchungen digitalisieren. Es gelten folgende Rahmenbedingungen: • Das Fitnessstudio bietet verschiedene Kurse an (z. B. Yoga, Pilates, Spinning, Zumba). Von jedem Kurs werden Bezeichnung, Wochentag, Uhrzeit und maximale Teilnehmeranzahl gespeichert. • Jeder Kurs wird von genau einem Trainer geleitet. Ein Trainer kann mehrere Kurse leiten. Von jedem Trainer werden Vorname, Nachname und Spezialgebiet erfasst. • Mitglieder können sich für beliebig viele Kurse anmelden. Für jeden Kurs können sich mehrere Mitglieder einschreiben. Bei jeder Anmeldung wird das Anmeldedatum gespeichert. Von jedem Mitglied werden Vorname, Nachname, E-Mail-Adresse und Beitrittsdatum erfasst.	
1.1. Aufgabe 1 – Entity-Relationship-Modell (ERM)	**Pun**
Erstelle auf Basis des Szenarios ein vollständiges ERM: Füge das fertige ERM als Bild in dieses Dokument ein. **(draw.io-Export)**	——

Name: Emir Alici **Datum:** 08.04.2026 **Klasse:** 3AHIF



1.2. Aufgabe 2 – Relationales Modell (RM)	**Pun**
Leite aus deinem ERM ein vollständiges Relationales Modell ab: Füge das fertige RM als Bild hier ein. **(draw.io-Export)**	

Name: Emir Alici **Datum:** 08.04.2026 **Klasse:** 3AHIF



1.3. Aufgabe 3 – Python-Skript mit SQLite3 ★ KI-Tools erlaubt

Pun

Schreibe ein Python-Skript Nachname_Vorname_fitnessstudio.py, das:

1. Eine SQLite-Datenbank Nachname_Vorname_fitnessstudio.db erstellt
2. Alle Tabellen aus dem RM anlegt
3. Sinnvolle Beispieldaten einfügt:
   ≥ 3 Trainer • ≥ 5 Kurse • ≥ 6 Mitglieder • ≥ 8 Anmeldungen

1.4. Aufgabe 4 – Anvil-Implementierung

Pun

Implementiere die Fitnessstudio-Verwaltung als Anvil-Webanwendung (anvil.works).

Pflichtfunktionen:

1. Kursübersicht – Alle Kurse inkl. Trainername

Kurs	Wochentag	Uhrzeit	Trainer	Teilnehmer	Anmelden
Morning Yoga	Montag	08:00	Berger Anna	2/15	Anmelden
Power Lifting	Dienstag	17:00	Huber Markus	2/10	Anmelden
Pilates Basics	Mittwoch	10:00	Mayer Sophie	2/12	Anmelden
HIIT Cardio	Donnerstag	18:30	Schneider Leon	1/20	Anmelden
Abend Yoga	Freitag	19:00	Berger Anna	2/15	Anmelden
Rückenfit	Samstag	09:00	Huber Markus	1/10	Anmelden

2. Neue Anmeldung

Mitglied	Anmelden
Bauer Thomas	Anmelden
Fuchs Stefan	Anmelden
Schwarz Maria	Anmelden
Weiß David	Anmelden
Braun Nina	Anmelden

Name: Emir Alici **Datum:** 08.04.2026 **Klasse:** 3AHIF

Bonusfunktionen (optional): • Mitglieder in einem Kurs anzeigen	
1.5. Aufgabe 5 - Abgabe	
1. Nachname Vorname in der Kopfzeile eintragen 2. GitHub-Link oben eintragen 3. Anvil Publish-Link oben eintragen 4. Folgende Dateien ins Git-Repo pushen a. Nachname_Vorname_ERM.draw.io b. Nachname_Vorname_RM.draw.io c. Nachname_Vorname_fitnessstudio.py 5. Code hier einfügen	

Alici_Emir_fitnessstudio.py

```
import sqlite3

# Verbindung zur SQLite-Datenbank (Datei wird erstellt, falls sie nicht existiert)
conn = sqlite3.connect(r"E:\Dbi_PLF\Alici_Emir_fitnessstudio.db")
cursor = conn.cursor()

cursor.executescript("""
DELETE FROM Trainer;
DELETE FROM Kurs;
DELETE FROM Mitglied;
DELETE FROM MeldetAn;
""")
# Tabellen erstellen
cursor.execute("""
CREATE TABLE IF NOT EXISTS Trainer (
    TrainerId INTEGER PRIMARY KEY,
    Vorname TEXT,
    Nachname TEXT,
    Spezialgebiet TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Kurs (
    KursId INTEGER PRIMARY KEY,
    Wochentag TEXT,
    Uhrzeit TEXT,
    Bezeichnung TEXT,
    Max_Teilnehmer INTEGER,
    TrainerId INTEGER,
    FOREIGN KEY (TrainerId) REFERENCES Trainer(TrainerId)
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Mitglied (
    MitgliedId INTEGER PRIMARY KEY,
    Beitrittsdatum DATE,
    Vorname TEXT,
    Nachname TEXT,
    Email TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS MeldetAn (
    MitgliedId INTEGER,
    KursId INTEGER,
    AnmeldeDatum DATE,
    PRIMARY KEY (MitgliedId, KursId),
```

Name: Emir Alici **Datum:** 08.04.2026 **Klasse:** 3AHIF

```
    FOREIGN KEY (MitgliedId) REFERENCES Mitglied(MitgliedId),
    FOREIGN KEY (KursId) REFERENCES Kurs(KursId)
)
""")

# Beispieldaten einfügen

# ≥ 3 Trainer
trainer = [
    (1, "Anna", "Müller", "Yoga"),
    (2, "Max", "Schmidt", "Krafttraining"),
    (3, "Laura", "Weber", "Cardio")
]
cursor.executemany("INSERT INTO Trainer VALUES (?, ?, ?, ?)", trainer)

# ≥ 5 Kurse
kurse = [
    (1, "Montag", "08:00", "Yoga Anfänger", 15, 1),
    (2, "Dienstag", "10:00", "Power Yoga", 12, 1),
    (3, "Mittwoch", "18:00", "Kraft Basics", 10, 2),
    (4, "Donnerstag", "17:00", "HIIT", 20, 3),
    (5, "Freitag", "09:00", "Cardio Mix", 18, 3)
]
cursor.executemany("INSERT INTO Kurs VALUES (?, ?, ?, ?, ?, ?)", kurse)

# ≥ 6 Mitglieder
mitglieder = [
    (1, "2024-01-10", "Tom", "Becker", "tom@example.com"),
    (2, "2024-02-15", "Lisa", "Klein", "lisa@example.com"),
    (3, "2024-03-20", "Paul", "Fischer", "paul@example.com"),
    (4, "2024-04-05", "Sara", "Wolf", "sara@example.com"),
    (5, "2024-05-01", "Jan", "Hoffmann", "jan@example.com"),
    (6, "2024-06-12", "Nina", "Schulz", "nina@example.com")
]
cursor.executemany("INSERT INTO Mitglied VALUES (?, ?, ?, ?, ?)", mitglieder)

# ≥ 8 Anmeldungen
anmeldungen = [
    (1, 1, "2024-07-01"),
    (2, 1, "2024-07-02"),
    (3, 2, "2024-07-03"),
    (4, 3, "2024-07-04"),
    (5, 4, "2024-07-05"),
    (6, 5, "2024-07-06"),
    (1, 3, "2024-07-07"),
    (2, 4, "2024-07-08")
]
cursor.executemany("INSERT INTO MeldetAn VALUES (?, ?, ?)", anmeldungen)

# Änderungen speichern und Verbindung schließen
conn.commit()
conn.close()

print("Datenbank erfolgreich erstellt und mit Beispieldaten gefüllt.")
```

Anvil Frontend Code

```
from ._anvil_designer import Kurs_DatenTemplate

from anvil import *

import anvil.tables as tables

import anvil.tables.query as q

from anvil.tables import app_tables

import anvil.server
```

Name: Emir Alici **Datum:** 08.04.2026 **Klasse:** 3AHIF

```
class Kurs_Daten(Kurs_DatenTemplate):
  def __init__(self, **properties):
    # Set Form properties and Data Bindings.
    self.init_components(**properties)

    anvil.server.call('get_anmelde_daten')
from ._anvil_designer import AnmeldungTemplate
from anvil import *
import anvil.tables as tables
import anvil.tables.query as q
from anvil.tables import app_tables
import anvil.server

class Anmeldung(AnmeldungTemplate):
  def __init__(self, **properties):
    # Set Form properties and Data Bindings.
    self.init_components(**properties)

    anvil.server.call('get_kunde')
    # Any code you write here will run before the form opens.
```

Anvil Backend Code

```
import anvil.tables as tables
import anvil.tables.query as q
from anvil.tables import app_tables
import anvil.files
from anvil.files import data_files
import anvil.server
import sqlite3

@anvil.server.callable
def get_kurs_daten():
  with sqlite3.connect("Alici_Emir_fitnessstudio.db") as conn:
```

Name: Emir Alici **Datum:** 08.04.2026 **Klasse:** 3AHIF

```
  cursor = conn.cursor()


  query = """
    SELECT Kurs.Bezeichnung ,Kurs.Wochentag,Kurs.Uhrzeit, Trainer.Vorname || ' ' || Trainer.Nachname AS
Name,Kurs.Max_Teilnehmer
    FROM Kurs
    LEFT JOIN Trainer
    ON Kurs.TrainerId = Trainer.TrainerId
    """
  cursor.execute(query)
  result = cursor.fetchall()
  return result


@anvil.server.callable
def get_kunde():
 with sqlite3.connect("Alici_Emir_fitnessstudio.db") as conn:
  cursor = conn.cursor()
  query = """SELECT Vorname || Nachname AS name FROM Mitglied"""
  cursor.execute(query)
  result = cursor.fetchall()
 return result
```